MBEYA UNIVERSITY OF SCIENCE AND TECHNOLOGY

COLLEGE: INFORMATION AND COMMUNICATION TECHNOLOGY

DEPARTMENT: COMPUTER SCIENCE AND ENGINEERING

PROGRAM: BACHELOR OF COMPUTER SCIENCE

COURSE NAME: APPLICATION DEVELOPMENT FOR MOBILE DEVICES

COURSE CODE: CS 8306

LEVEL: THIRD YAER

LECTURER NAME: MR. AMANI RUKOIJO

TASK NAME: GROUP ASSIGNEMENT

GROUP MEMBERS **(FundiOnDemand)**

S/NO	NAME	REG NO	CA NO
1.	AGATHA JONAS	23100533350051	CA/BCS/25/11923
2.	ALTO DEZDEL	23100533350059	CA/BCS/25/11924
3.	NICOMED CORNEL	23100533350050	CA/BCS/25/13403
4.	TEMI SIMEO	23100533350091	CA/BCS/25/2758

PROJECT TITLE

MUST EVENT AND CONFERENCE APPLICATION SYSTEM

INTRODUCTION

Many universities organize events and conferences where people need to register and get information like the date, place, and presenter. Sometimes this is done using paper or simple lists, which can cause problems such as losing information, making mistakes, or taking too much time. Because of these challenges, we developed the MUST Event and Conference Management System. This system allows users to register, log in, view event details, and for administrators to add new events. All information is stored safely in a computer database. The system works online as a website and can also work offline because it is a Progressive Web Application (PWA). This makes event management easier, faster, and more organized.

Problem Description

Mbeya University of Science and Technology (MUST) regularly organizes academic conferences, workshops, seminars, student government meetings, innovation exhibitions, and other institutional events. These events play an important role in academic growth, networking, knowledge sharing, and institutional development.

However, most event management and communication processes at the university are handled manually or through informal communication channels such as posters, notice boards, WhatsApp groups, and social media announcements. While these methods provide temporary communication, they lack centralization, structure, and long-term record keeping.

This project therefore focuses on designing and implementing a Progressive Web Application (PWA) that can manage MUST events and conferences in a structured and digital manner.

Motivation

The motivation behind this project is both practical and academic.

From a practical perspective, the system aims to solve a real institutional problem by providing a centralized event management platform that is accessible anytime and anywhere through mobile devices.

From an academic perspective, this project demonstrates the application of full-stack mobile application development concepts studied in the course, including:

- Frontend development
- Backend development

- Database integration
- Authentication and security implementation
- Responsive mobile-first design
- Progressive Web Application (PWA) features

Objectives

Main Objective

To develop MUST event and Conference application system

Specific Objectives

1. To develop a secure login and registration system
2. To allow users to create and view events
3. To store event data in a structured database
4. To implement authentication and access control
5. To build a Progressive Web Application (PWA)

SYSTEM REQUIREMENTS

System requirements describe what the system needs in order to work properly. These include both hardware and software requirements.

Functional Requirement

It describes what the system can do. The system must include

1. Allow users to register
2. Allow users to login
3. Allow Coordinator to upload events
4. Allow admin to manage the system
5. Allow users to view events
6. Store user and event data
7. Support offline caching (PWA)

Non-functional Requirements

It describes how the system should work. The system must include

1. System must be secure
2. System must be responsive (mobile-friendly)
3. System must load quickly
4. System must support multiple users
5. System must ensure data integrity

SYSTEM ARCHITECTURE

Introduction

The MUST Event and Conference Management System was developed to solve this problem. It allows users to register, log in, and view event details online. Administrators can also add and manage events easily. The system stores all information safely in a database and can even work offline because it is a Progressive Web Application (PWA). This makes event management faster, easier, and more organized.

The frontend is the part users see and interact with. It is built using HTML, CSS, and JavaScript. Users use the frontend to register, log in, and view or upload events.

The backend is developed using PHP. It receives requests from the frontend, processes the data, and communicates with the database. For example, when a user logs in, the backend checks the username and password in the database.

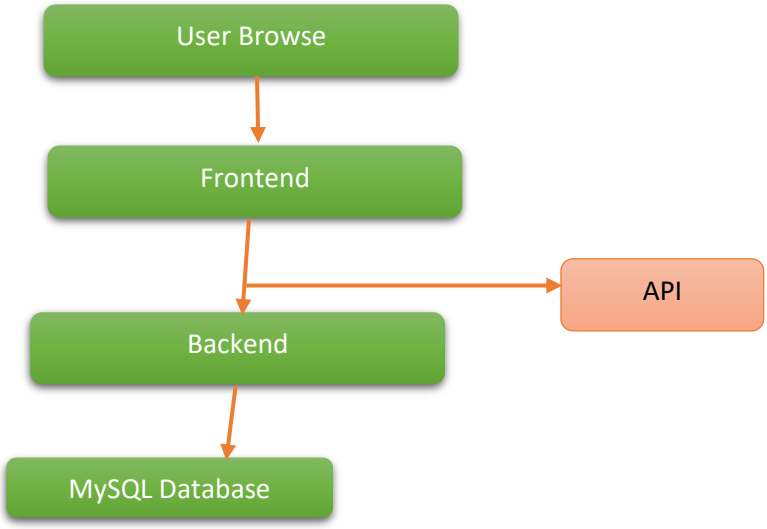
The database is created using MySQL. It stores all important information such as user details and event information. The backend retrieves and saves data in this database.

The system also includes Progressive Web Application (PWA) features. A service worker stores important files in the browser so the system can still open even without internet connection.

Overall System Design

The application follows a 3-tier architecture: a PWA Frontend (HTML, CSS, JS, Bootstrap), Backend (PHP) for Logic Layer, and a Database (MySQL)

Client-Server Architecture Diagram



Database Schema (ERD)

Database include users table and events table

Users Table

- id (Primary Key)
- username
- email
- password
- role

Events Table

- id (Primary Key)
- title
- description
- date
- location

Authentication Mechanism

- Login using username & password
- Password stored using hashing
- Session or token-based authentication
- Role-based access control

Security Measures Used

Implementation of password hashing and frontend Session Guards to prevent unauthorized page access and Input validation

Authentication Mechanism

The system uses a username and password-based authentication process. During login, the user submits credentials through the frontend interface. The frontend sends a POST request to the backend, where the credentials are validated against the stored records in the MySQL database. If the credentials are correct, a server-side session is created and the user is redirected to the dashboard. If incorrect, access is denied and an error message is displayed.

This ensures that only registered users can access protected system features.

Security Measures Used

Implementation of password hashing and frontend Session Guards to prevent unauthorized page access and Input validation

IMPLEMENTATION DETAILS

Technologies Used

- Frontend includesHTML5, CSS3, JavaScript, Service Worker (PWA) and Manifest
- Backend uses PHP
- Database uses MySQL

Frontend Implementation

Developed using a sidebar-layout template with dynamic JavaScript fetch calls to populate event cards. Responsive login and register pages, PWA manifest.json, Service-worker.js for offline support, Fetch API for communication with backend

Backend Implementation

REST-style PHP scripts that process JSON requests and interface with the MySQL database.

Database design

This includes Relational database, Primary and foreign keys and Indexed columns for performance

Challenges faced

Connecting frontend to backend, Service worker caching errors and JSON data handling

Assumptions made

Users have a stable internet connection for initial PWA installation and data synchronization. Admin is trusted and Database server is secure

SECURITY CONSIDERATIONS

Security is a critical component of the MUST Event and Conference Application System because the system handles user credentials and event management operations. Proper authentication and authorization mechanisms were implemented to protect user data and restrict unauthorized access.

- Authentication Mechanism

The system uses a username and password-based authentication process. During login, the user submits credentials through the frontend interface. The frontend sends a POST request to the backend, where the credentials are validated against the stored records in the MySQL database. If the credentials are correct, a server-side session is created and the user is redirected to the dashboard. If incorrect, access is denied and an error message is displayed.

This ensures that only registered users can access protected system features.

- Password Security (Hashing)

Passwords are never stored in plain text. During registration, the password is securely hashed using PHP's password_hash() function before being stored in the database. During login, the system verifies the password using password_verify().

Hashing ensures that even if the database is compromised, raw passwords cannot be retrieved, significantly improving system security.

- Session Management

After successful login, the system creates a secure server-side session. The session stores user identification and role information (Admin or User). Each protected page verifies whether a valid session exists before granting access. If no session is found, the user is automatically redirected to the login page.

This prevents unauthorized users from accessing restricted pages.

- Role-Based Access Control (Authorization)

The system implements role-based authorization with two roles:

Admin: Can create, update, and delete events.

User: Can only view events.

The user's role is stored in the database and checked during each restricted action. Both frontend and backend enforce role verification to ensure proper access control.

- Protection Against SQL Injection

To prevent SQL injection attacks, prepared statements are used for all database queries. This ensures that user input is treated strictly as data and not executable SQL code.

- Input Validation

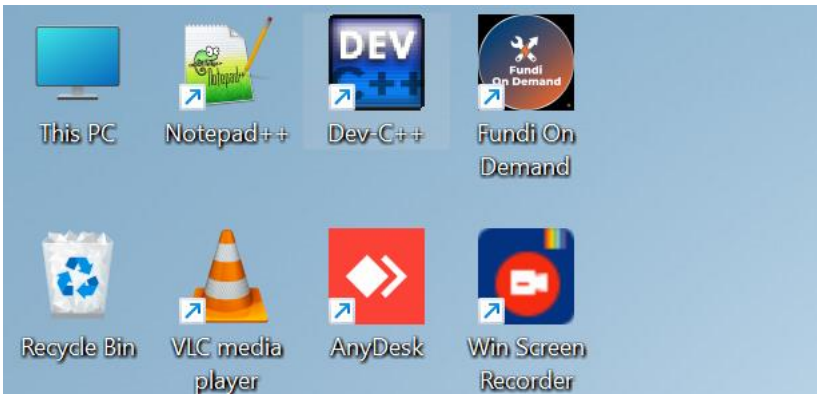
Validation is performed at both frontend and backend levels to ensure data integrity. Required fields must be completed, and improper or malicious inputs are rejected before database insertion.

TESTING & RESULTS

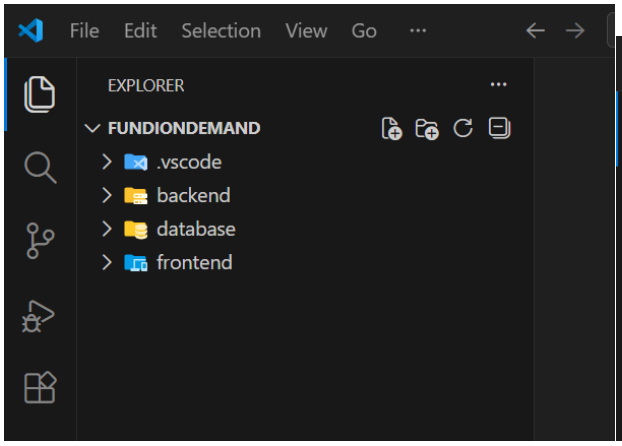
Screenshots of the application

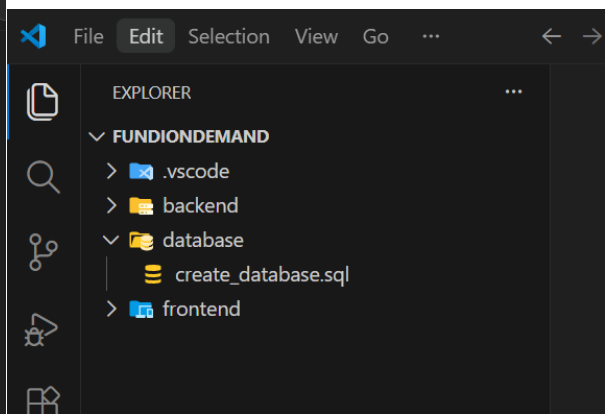
The following images show different application interactions.

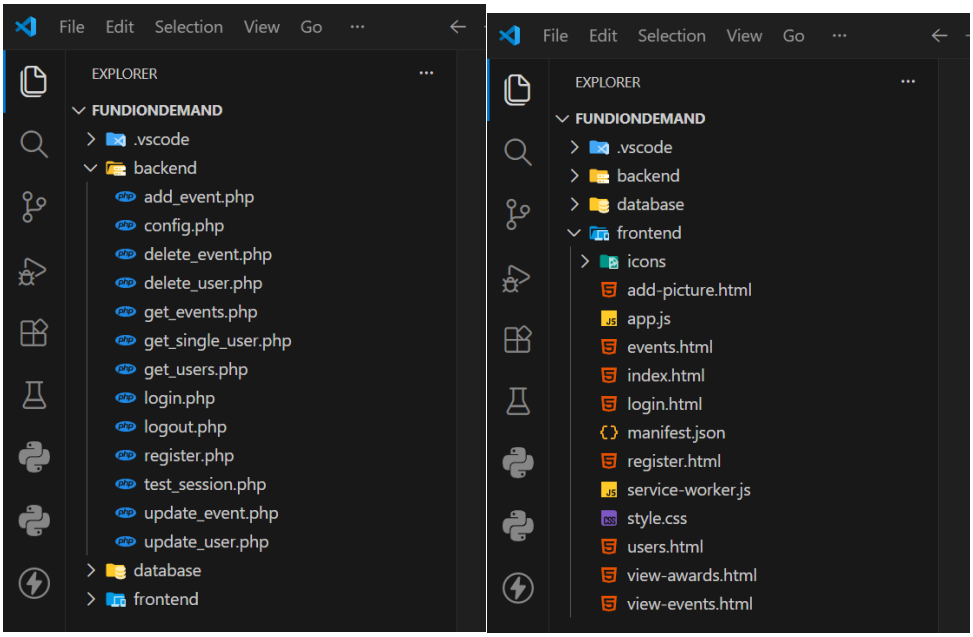
Logo of the app (installed application icon which is FundiOnDemand)



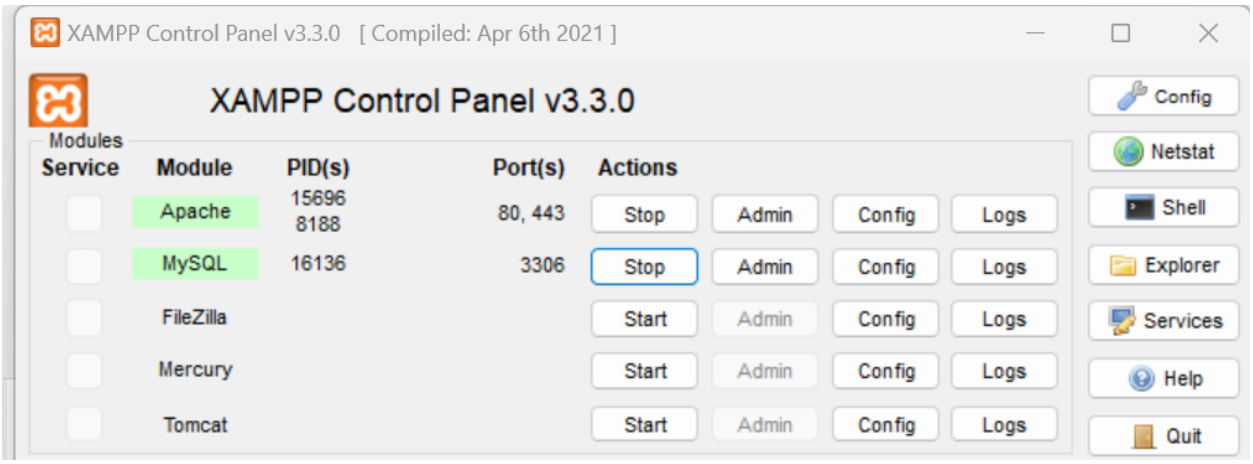
Project structures and files arrangement using visual studio



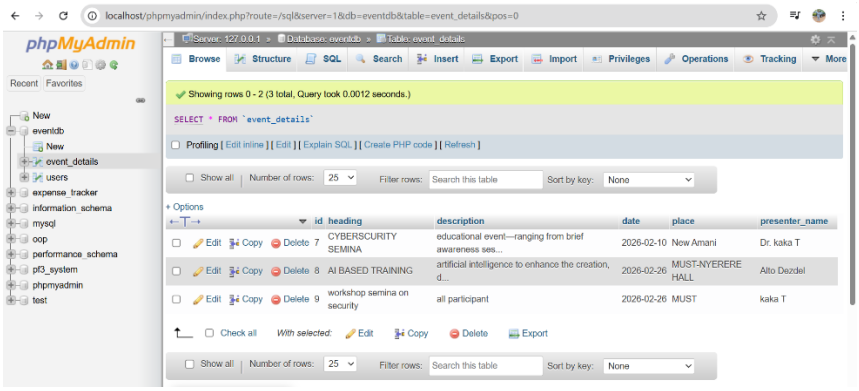




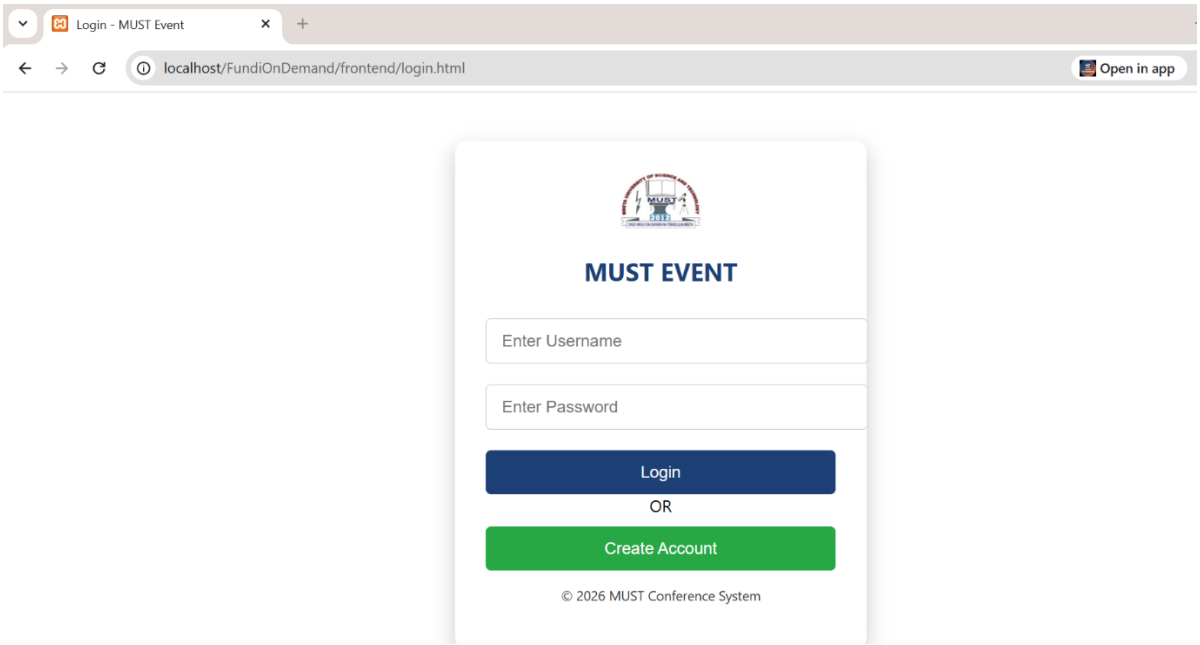
Server apache (Xampp)



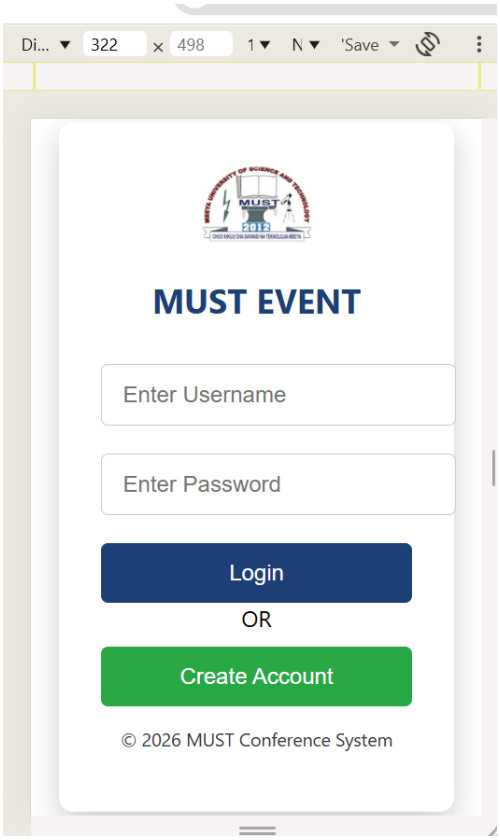
Database (Xampp)



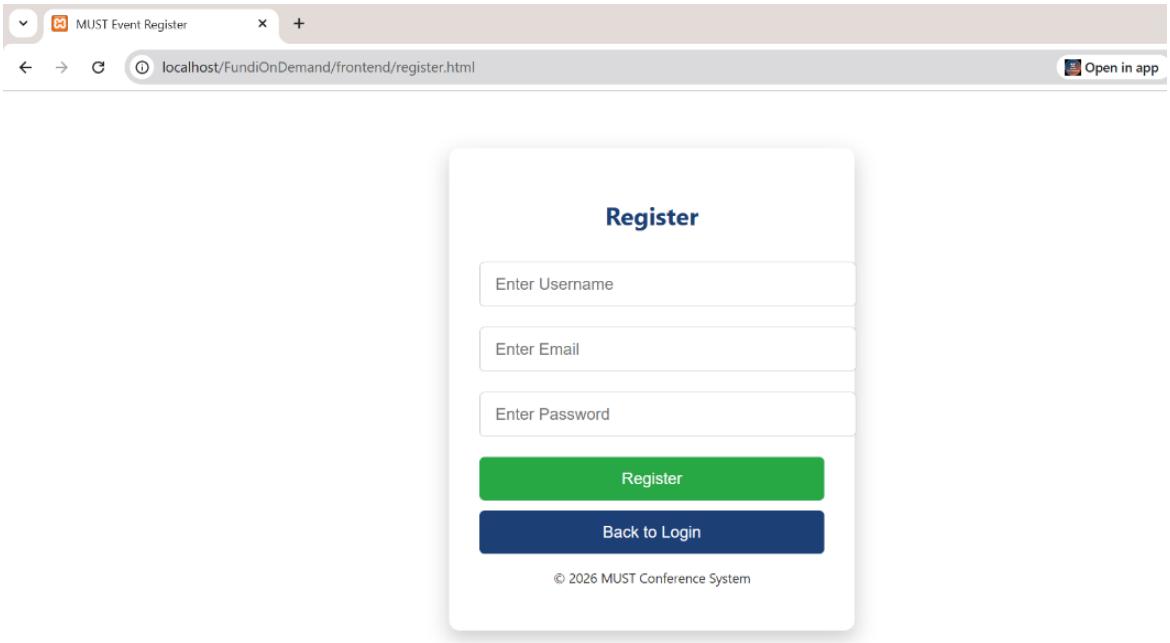
Login interface for computers



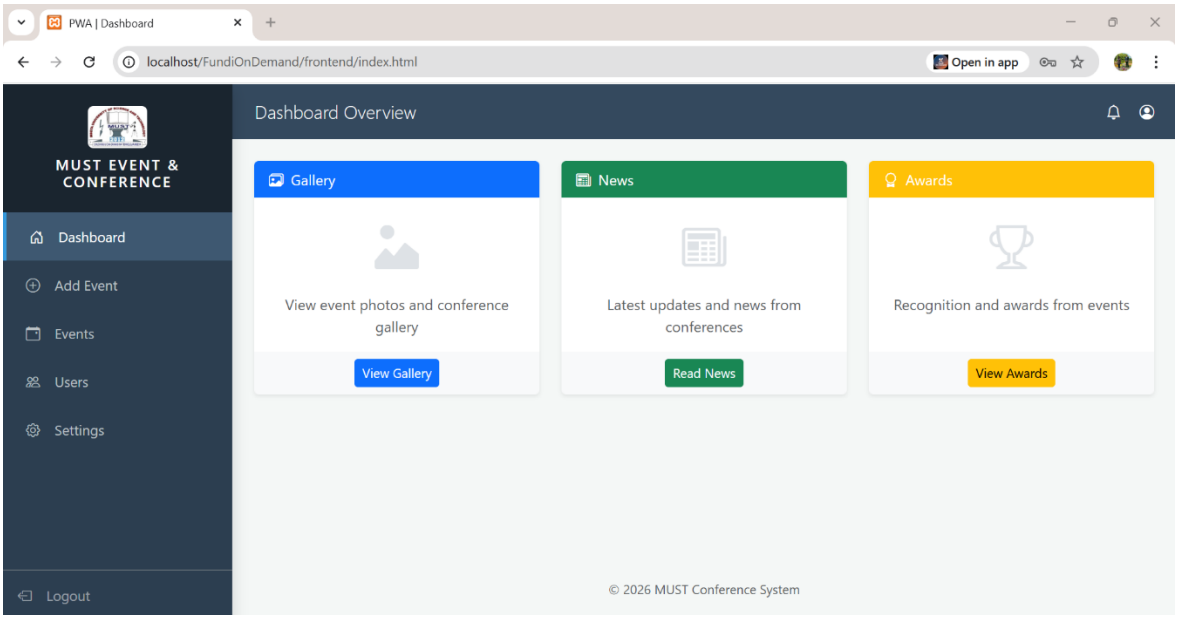
Login view for mobile devices



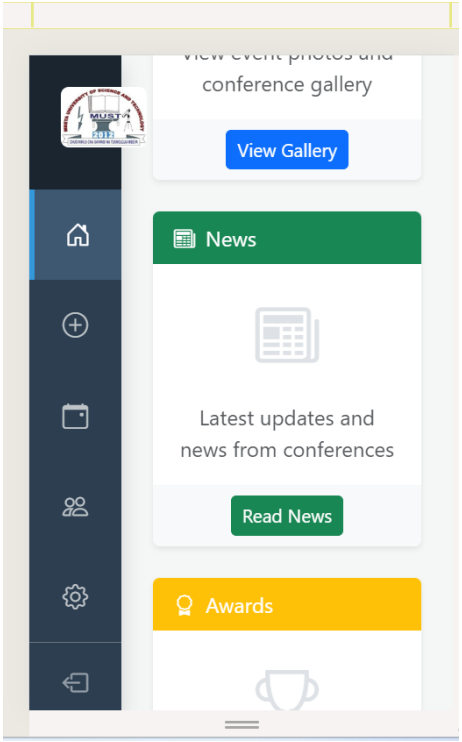
Register interface



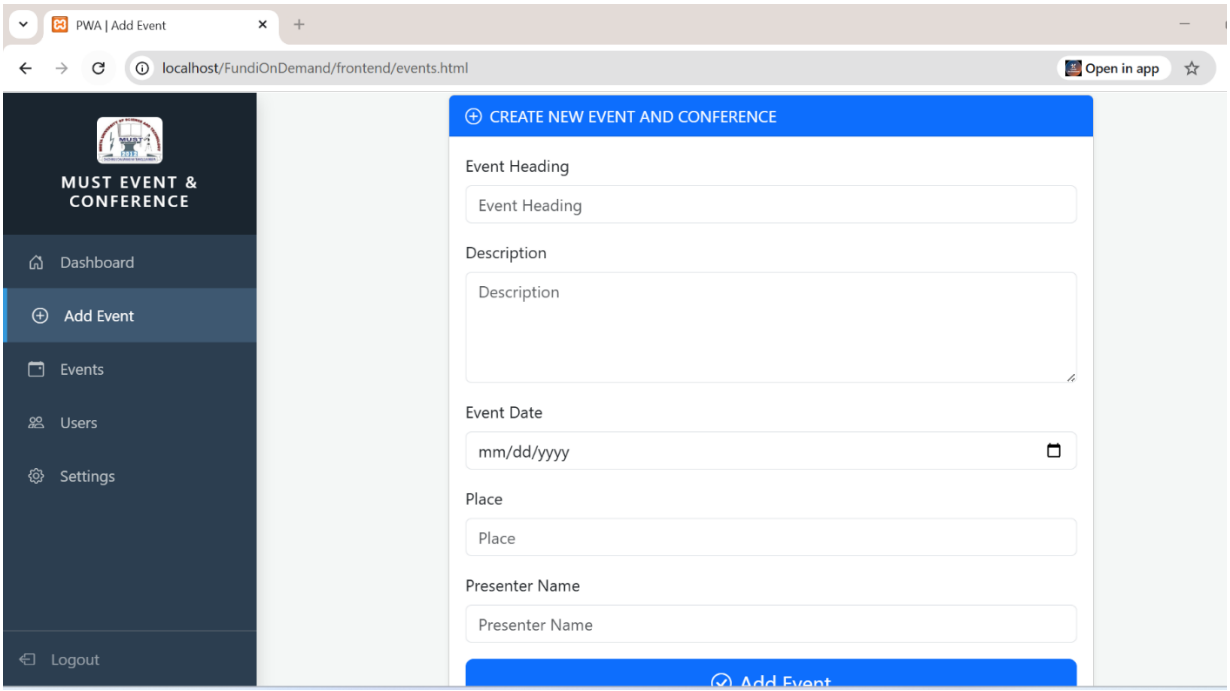
Dashboard view for computers



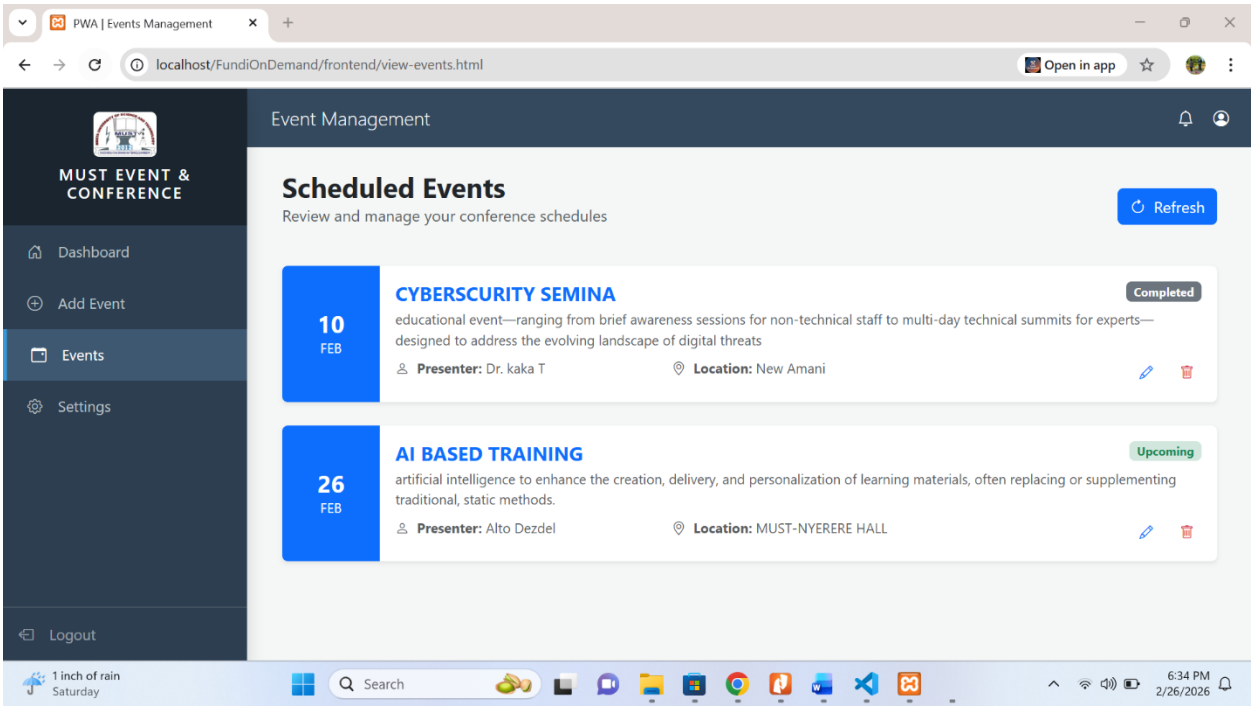
Dashboard for view from Mobile devices



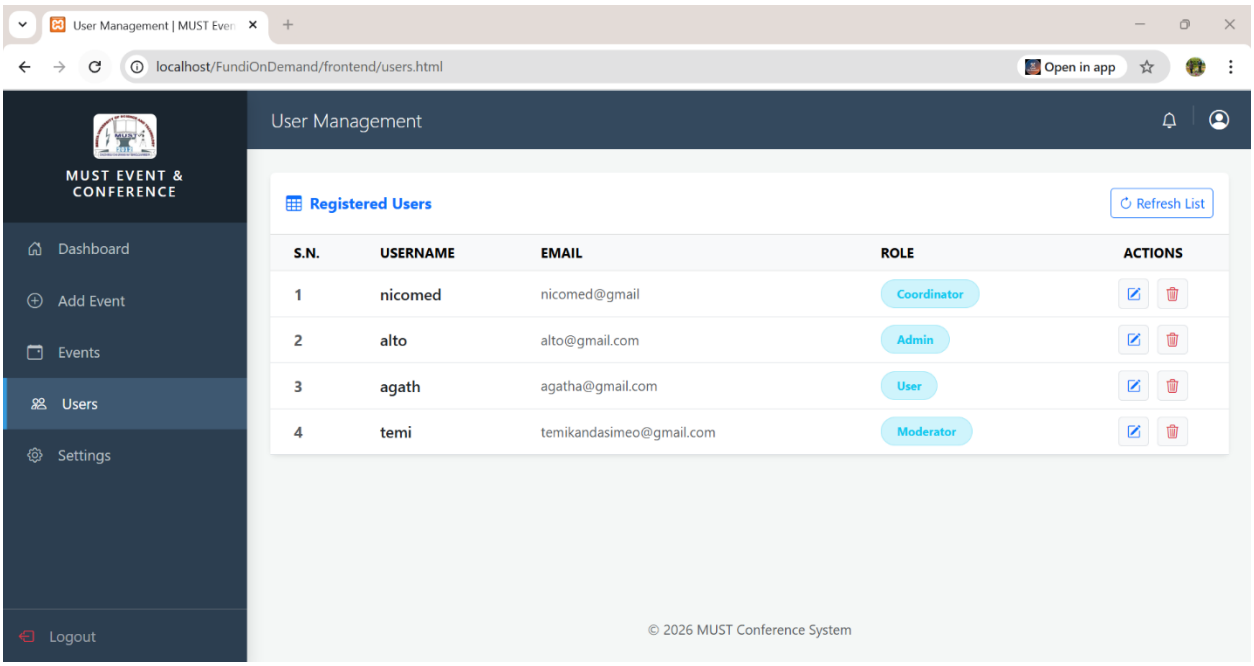
Add event



View event



Users view



EXPLANATION OF MAJOR FEATURES

login works

Below is the working of the login to our project

- The user enters username and password.
- The user clicks the Login button.
- The system sends the information to the server (backend).
- The server checks the database:

If the username exists and if the password is correct

If correct → the user is logged in and redirected to the dashboard.

If incorrect → an error message is shown.

User → Server → Database → Response → User

Offline Mode Works (PWA)

- The first time a user opens the website:

The Service Worker saves important files (HTML, CSS, JS).

- If the internet connection is lost:

The system loads the saved files from the browser.

- The website can still open even without internet.

Event Upload Works

- The admin fills in

Title, Description, date and location

- The admin clicks Upload.
- The system sends the data to the server.
- The server saves the event in the database.
- The event appears on the events page.

Form → Server → Database → Displayed on page

Incomplete Features (If Any)

Example such as email verification not implemented and advanced analytics not implemented

FUTURE WORK

Possible Improvements

Integration of a real-time chat system for conference attendees to network. Also add email verification, add payment integration, add event reminder notifications and add admin analytics dashboard

Scalability Considerations

Migrating to a cloud-hosted database for higher traffic volume, use load balancing, improve database indexing and add REST API structure

CONCLUSION

The MUST Event and Conference Application System was successfully designed and implemented as a full-stack Progressive Web Application. The system demonstrates frontend interface development using HTML, CSS, Bootstrap, and JavaScript. Backend logic implementation using PHP, Secure authentication using password hashing and sessions, Full CRUD functionality integrated with MySQL database, Responsive mobile-first design, PWA functionality with offline caching.

The project meets all defined objectives and satisfies the requirements of full-stack mobile application development as outlined in the course.

This project also provided practical experience in integrating multiple technologies to build a secure, scalable, and user-friendly application.